Implement the K-Nearest Neighbors (KNN) algorithm for image classification using the Fashion MNIST dataset. Experiment with different values of K and analyze their impact on model performance.

Tasks:

● Load and preprocess the Fashion MNIST dataset.

● Implement KNN for multi-class classification.

● Experiment with different values of K and evaluate performance.

● Discuss the impact of different K values on model accuracy and computational efficiency.

Objective

- To implement **KNN for multi-class image classification**
- To experiment with different values of **K**
- To analyze impact on:
    - Accuracy
    - Computational cost

Dataset: Fashion MNIST (No need to write in rough record)

- 70,000 grayscale images (28×28 pixels)
- 10 classes:

Label	Class
0	**T-shirt/top**
1	**Trouser**
2	**Pullover**
3	**Dress**
4	**Coat**
5	**Sandal**
6	**Shirt**

LabelClass

7 Sneaker

8 Bag

9 Ankle boot

Algorithm

1. Load dataset
2. Preprocess (flatten + normalize)
3. Train KNN
4. Evaluate performance
5. Compare different K values

Program

```
# ------------------------------
# 1. Import Libraries
# ------------------------------
import numpy as np
from tensorflow.keras.datasets import fashion_mnist
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler

# ------------------------------
# 2. Load Dataset
# ------------------------------
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

# ------------------------------
# 3. Preprocessing
# ------------------------------
# Flatten images (28x28 → 784)
X_train = X_train.reshape(len(X_train), -1)
```

```
X_test = X_test.reshape(len(X_test), -1)


# Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# ------------------------------
# 4. Experiment with different K
# ------------------------------
k_values = [1, 3, 5]

for k in k_values:
    print(f"\n===== K = {k} =====")

    knn = KNeighborsClassifier(n_neighbors=k, n_jobs=-1)
    knn.fit(X_train[:10000], y_train[:10000])  # subset for speed
    y_pred = knn.predict(X_test[:2000])
    acc = accuracy_score(y_test[:2000], y_pred)
    print("Accuracy:", acc)
    print(classification_report(y_test[:2000], y_pred))
```